
Informe de Trabajo Práctico 0

Cuartetos Primos - Optimización de Código

Materia: Teoría de Algoritmos
Cátedra: Echevarria
Cuatrimestre: Segundo Cuatrimestre

Alumno: Francisco Oscar Ballerio
Padrón: 114777

Fecha de Entrega: 29 de Agosto de 2026

Índice

1. Código Inicial	2
1.1. Supuestos y Condiciones del Algoritmo	2
1.2. Limitaciones en el algoritmo dado	2
2. Diseño y Mejora del Algoritmo	3
2.1. Optimizando <code>esprimo</code>	3
2.2. Optimizando <code>prog</code>	4
3. Pseudocódigo y Estructuras de Datos	4
3.1. <code>esprimo</code>	4
3.2. <code>prog</code>	5
4. Complejidad Temporal	5
4.1. Original	5
4.2. Optimizado	5
5. Comparación de Tiempos de Ejecución	5
5.1. Base	6
5.2. Optimizado	7
5.3. Conclusión	8
6. Bibliografía	8

1. Código Inicial

La solución al ejercicio está compuesta por dos funciones: **esprimo** y **prog**.

- **esprimo** recibe un número entero n y busca la cantidad de divisores de este en el conjunto de números enteros $[2, n - 1]$ recorriendo cada elemento. Si esta cantidad es 0 significa que el número es primo y, de tener al menos 1 divisor, el número no es primo.
- **prog** mide y muestra el tiempo de ejecución requerido para hallar e imprimir cuartetos de números primos dentro de una misma decena, iterando desde 11 hasta 1.000.000 y validando cada candidato con **esprimo**.

1.1. Supuestos y Condiciones del Algoritmo

Para que el algoritmo funcione, se necesita:

- Que el límite (n) sea un entero positivo.

El algoritmo:

- Busca todos los cuartetos de primos en la misma decena en el intervalo $[2, n - 1]$.
- Todos los elementos de la solución serán menores o iguales que el límite.
- Los tiempos de ejecución dependen del hardware y de la cantidad de tareas que el OS está ejecutando al momento de correr el código.
- La versión de Python utilizada es la 3.10.

1.2. Limitaciones en el algoritmo dado

esprimo:

- Recorre todos los números hasta $n - 1$.
- El bucle **while** es menos eficiente que un **for**.
- Cuenta la cantidad de divisores que tiene el número.
- Compara si la cantidad de divisores es 0 para devolver algo cuando podría devolver directamente la comparación, pues devuelve un **bool**.

prog:

- El bucle recorre todos los números de 11 a 1000000. Lo que genera:
 - Se saltea el primer cuarteto de primos.
 - Chequea si un número es primo varias veces.
 - Sobrepassa la decena.
 - Cada vez que se encuentra un cuarteto, se imprime. Lo que genera operaciones de I/O que ralentizan la búsqueda de los mismos. Imprimir causa que el OS tenga que manejar operaciones de I/O, llamadas al kernel y cambios de contexto.

- Ejemplo:
 - $i = 11 \rightarrow \text{esPrimo}(11) \wedge \text{esPrimo}(13) \wedge \text{esPrimo}(17) \wedge \text{esPrimo}(19)$
 - $i = 13 \rightarrow \text{esPrimo}(13) \wedge \text{esPrimo}(15) \wedge \text{esPrimo}(19) \wedge \text{esPrimo}(21)$

2. Diseño y Mejora del Algoritmo

2.1. Optimizando esprimo

1. De **while** pasaremos a usar **for**, que ejecuta la iteración en C. Mientras que **while** verifica en Python la condición en cada iteración.
2. De la mano del cambio del **for**, agregamos un **range(inicio, fin, salto)** que devuelve una lista con el intervalo $[inicio, fin]$.
3. Buscaremos divisores entre 2 y $\sqrt{n}$, pues si $n = a \cdot b$ entonces alguno de los dos es menor o igual a $\sqrt{n}$, pues $\sqrt{n} \cdot \sqrt{n} = n$. En este caso lo que queremos es encontrar al menos 1 divisor para n , entonces si en $[1, \sqrt{n}]$ no encontramos ninguno, en $[\sqrt{n}, n - 1]$ tampoco habrá porque si a, b alguno es mayor a $\sqrt{n}$, entonces necesariamente el otro debe ser menor a $\sqrt{n}$.

De la mano de este cambio viene el uso de la biblioteca integrada **math** para calcular la raíz de n , que implementa el cálculo y el casteo a **int** en C.

4. Si al menos un número de $[1, \sqrt{n}]$ divide a n , entonces no es primo. Por eso, al encontrar un divisor cortamos el bucle.
5. División optimizada:

Cualquier número entero mayor a 3 puede representarse algebraicamente bajo una de las formas $6k$, $6k + 1$, $6k + 2$, $6k + 3$, $6k + 4$ o $6k + 5$ (para $k \geq 1$):

- Las formas $6k$, $6k + 2$ y $6k + 4$ son pares (divisibles por 2), por lo que son números compuestos.
- La forma $6k + 3$ es divisible por 3, siendo también compuesta.
- Las únicas formas restantes son $6k + 1$ y $6k + 5$, donde $6k + 5$ equivale algebraicamente a $6(k + 1) - 1$, quedando ambas cubiertas bajo el patrón $6k \pm 1$.

Dado que los números divisibles por 2 o 3 se pueden descartar en una verificación inicial (en este caso los obviamos por conveniencia del algoritmo), para determinar la primalidad basta con evaluar únicamente los divisores de la forma $6k \pm 1$ hasta el límite $\sqrt{n}$ [1].

Esto nos permite optimizar el algoritmo **esprimo** y reducir significativamente la cantidad de operaciones necesarias. Finalmente, lo mejor que se pudo mejorar la complejidad de este algoritmo es $O(\sqrt{n})$.

2.2. Optimizando prog

1. Restricción del espacio de búsqueda para cuartetos de primos:

Un cuarteto de números primos es un conjunto de la forma $\{p, p + 2, p + 6, p + 8\}$. Todo cuarteto con $p > 5$ cumple con las siguientes propiedades aritméticas ($\{5, 7, 11, 13\}$ es el único que no cumple, pero como no están en la misma decena no nos importa):

- Ninguno de sus elementos puede ser divisible por 2, 3 o 5.
- En aritmética modular, esto restringe la posición del primer elemento a una única clase de congruencia módulo 30 ($2 \times 3 \times 5 = 30$), específicamente $p \equiv 11 \pmod{30}$.

Como consecuencia, la distancia mínima entre el inicio de dos cuartetos consecutivos $\{p, \dots\}$ y $\{q, \dots\}$ es $q - p = 30$, y toda separación entre candidatos válidos debe ser forzosamente un múltiplo de 30 [2].

Esta propiedad permite optimizar el algoritmo de búsqueda, descartando la evaluación de valores intermedios y avanzando las iteraciones en saltos de 30 (o evaluando únicamente candidatos $p \equiv 11 \pmod{30}$).

2. Para optimizar el imprimir por pantalla agrupamos los candidatos en una lista de cuatruplas $(i, i + 2, i + 6, i + 8)$ para imprimir los cuartetos juntos luego de encontrarlos *todos*.
3. El crear la lista de cuartetos *inline* aumenta la performance del código. La implementación de este método está en CPython y evita usar `.append()` repetidas veces.

3. Pseudocódigo y Estructuras de Datos

▪ Lista:

- En la función `prog`, `resultados` es una lista con los cuartetos a imprimir.

▪ Tupla:

- Los cuartetos son almacenados como `tuple[int, int, int, int]` en la lista de resultados.

3.1. esprimo

```

recibo numero
limite = raiz_cuadrada(numero)
para cada num (desde 5, hasta limite, incrementando 6) {
    si resto(n / num) == 0 o si resto(n / (num + 2)) == 0 {
        devuelvo Falso
    }
}
devuelvo verdadero

```

3.2. prog

```

resultados = new lista[tupla(int, int, int, int)]
para cada i (desde 11, hasta 1000000, incrementando 30) {
    si esprimo(i) y esprimo(i + 2) y esprimo(i + 6) y esprimo(i + 8) {
        lista.agregar(tupla(i, i+2, i+6, i+8))
    }
}
para cada tupla de resultados {
    imprimir tupla
}
imprimir tiempo de ejecucion

```

4. Complejidad Temporal

4.1. Original

esprimo: loopea desde 2 hasta $n - 1$, es decir, tiene una complejidad de $O(n)$.

prog: tiene una complejidad de $O(n^2)$, ya que llama a **esPrimo** para cada número en el rango $[11, 1000000]$.

4.2. Optimizado

esprimo: el algoritmo recorre de 6 en 6 los números desde el 5 hasta $\sqrt{n}$. O sea, se realizan una cantidad de $\sqrt{n}/6$ operaciones, lo que da $O(\sqrt{n}/6)$ operaciones, que es lo mismo que $O(\sqrt{n})$.

prog: gracias a que **esprimo** mejoró su complejidad, este algoritmo también lo hizo. Pero la base de la complejidad sigue siendo la misma, pues la cantidad de llamadas a **esprimo** depende de n . Finalmente se realizan $n/30 \cdot 4$ llamadas a **esprimo**:

$$O(n/30) \cdot 4 \cdot O(\sqrt{n}) = O(n) \cdot O(\sqrt{n}) = O(n^{3/2})$$

5. Comparación de Tiempos de Ejecución

En la teoría conocemos la complejidad de ambos algoritmos. Ahora queremos comprobar empíricamente que estas complejidades sean correctas. Por lo tanto usaremos el paper proporcionado por la cátedra para aproximar por cuadrados mínimos y verificar que el error sea relativamente bajo.

5.1. Base

Cuadro 1: Tiempos de ejecución medidos para la versión original (base.py).

N	Tiempo promedio (s)
499	0.0088
626	0.0155
785	0.0245
984	0.0347
1234	0.0576
1547	0.0928
1939	0.1528
2430	0.2431
3046	0.3582
3818	0.5568
4786	0.8202
5999	1.2352

Cuadro 2: Ajuste por cuadrados mínimos (base.py).

Algoritmo	Modelo	c_1	c_2	Error cuadrático
busqueda_base	$c_1 n^2 + c_2$	$3,47 \times 10^{-8}$	$1,48 \times 10^{-2}$	$4,12 \times 10^{-3}$

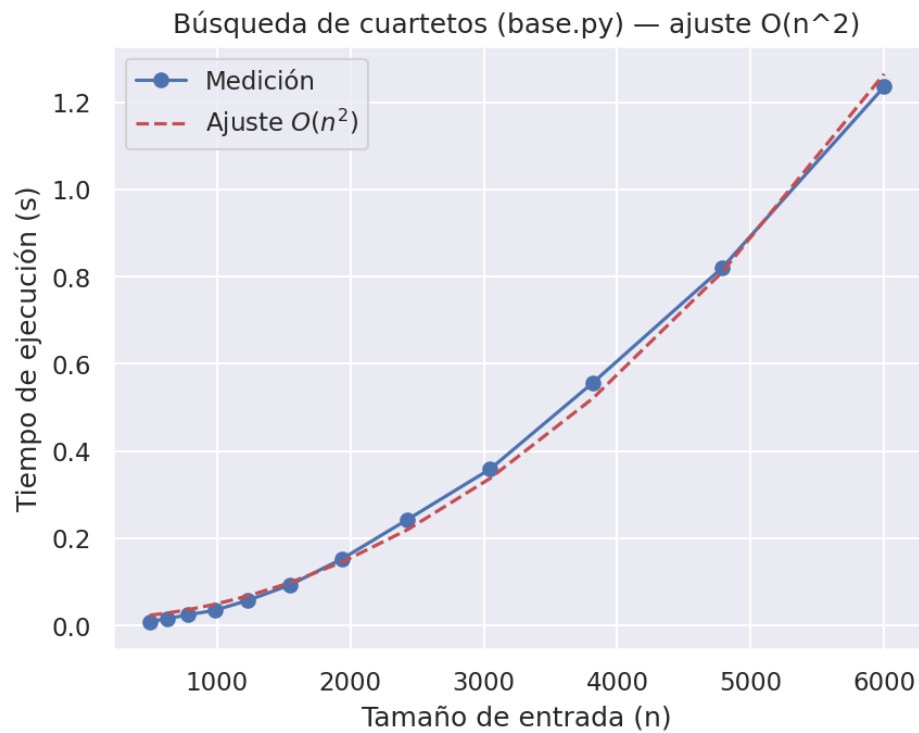


Figura 1: Búsqueda de cuartetos (base.py) — ajuste $O(n^2)$.

5.2. Optimizado

Cuadro 3: Tiempos de ejecución medidos para la versión optimizada (tp.py).

N	Tiempo promedio (s)
100000	0.00535
142709	0.00939
203659	0.01466
290640	0.02446
414769	0.04034
591914	0.06298
844716	0.09753
1205487	0.16287
1720341	0.27145
2455084	0.45752
3503629	0.74358
4999999	1.19270

Cuadro 4: Ajuste por cuadrados mínimos (tp.py).

Algoritmo	Modelo	c_1	c_2	Error cuadrático
busqueda_optimizada	$c_1 n^{3/2} + c_2$	$1,08 \times 10^{-10}$	$1,46 \times 10^{-2}$	$2,67 \times 10^{-3}$

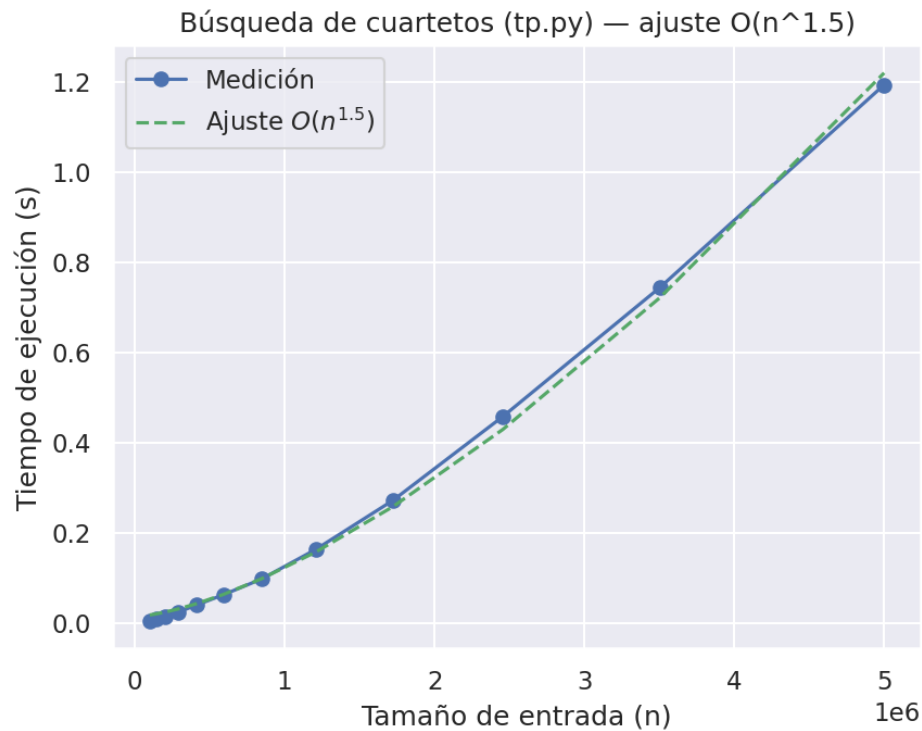


Figura 2: Búsqueda de cuartetos (tp.py) — ajuste $O(n^{1.5})$.

5.3. Conclusión

